Individual Technical Reflection Report: Smart Vacuum Robot — CEN425

Internet of Things: Applications and Networking

Amro Kamal Jalad
Department of Computer Engineering
Abu Dhabi University
Abu Dhabi, UAE
1092505@students.adu.ac.ae

***Abstract*—This document presents a personal reflection on the individual contribution to the CEN425 Smart Vacuum Robot project at Abu Dhabi University. The contribution covered software development, finite state machine design, camera integration, computer vision motion detection, web dashboard implementation, Telegram bot integration, and 3D chassis design using Autodesk Inventor Professional. The robot operates autonomously, navigating the perimeter of a 2m x 1m rectangular arena twice, using an HC-SR04 ultrasonic sensor to detect walls and an MPU6050 gyroscope to execute precise 90-degree right turns. The system features real-time remote monitoring through a live MJPEG camera stream with OpenCV motion detection, a multi-page web dashboard accessible from any device on the local network including mobile devices, and automated Telegram notifications including timed security photos sent every five seconds. A cleaning report feature was also implemented, allowing the user to view and download a PDF summary of all cleaning sessions completed during the day.**

***Index Terms*—Raspberry Pi 4, IoT, Autonomous Robot, Python, Finite State Machine, Camera, Web Dashboard, Telegram Bot**

I. Motivation

The Internet of Things is one of the fastest growing areas of engineering today, allowing physical devices to sense, communicate, and take action without the direct involvement of a person. The motivation for the CEN425 Smart Vacuum Robot project was the need to apply IoT principles to a meaningful and practical problem: automated home cleaning and remote monitoring. No longer a purely simulated system, the goal was to create a fully functional autonomous robot that could navigate a physical environment, identify obstacles, and supply real time visual feedback to a remote user via a web interface and messaging platform.

The driver for the software and system integration contribution was combining multiple independent subsystems into one complete integrated application. These subsystems encompass DC motor control and speed regulation using PWM, 90 degree turns using the gyroscope, ultrasonic wall detection, live camera streaming, computer vision motion detection using OpenCV, a responsive multi page web dashboard, and a Telegram security bot [4]. Each subsystem required individual research, implementation, and debugging before integration, making the project a genuine full stack embedded systems experience.

The need for real time remote access and control via a browser based dashboard, accessible from a phone or laptop without any additional software directly reflected the core IoT idea of connected and remotely accessible physical systems. Additionally, automatic Telegram photo alerts combined with computer vision motion detection provided an extra layer of usefulness to the project regarding security monitoring, demonstrating how IoT devices can serve purposes beyond their original intended function.

II. Personal Contribution

The individual effort of the CEN425 Smart Vacuum Robot project covered five specific areas: software and navigation code, finite state machine design, web dashboard and camera integration, Telegram bot and computer vision implementation, and 3D chassis design using Autodesk Inventor Professional. The fully assembled robot is shown in Fig. 1.



Fig. 1. Fully assembled Smart Vacuum Robot

A. Software and Navigation Code

All the navigation code for Python was written and implemented in a single Python script called `robot_final.py`. This included motor control using the lgpio library [2], reading data from the MPU6050 gyroscope [5] without requiring any external library, wall detection logic using the HC-SR04 ultrasonic sensor, and the entire mission with 2 laps and 8 turns. In order to avoid wheel slip affecting the gyroscope angle readings during forward motion, a soft start function was added that slowly increases the PWM duty cycle from zero to the target speed.

B. Finite State Machine Design

A Finite State Machine dictates the behavior of the robot and was designed prior to implementation of any code. To ensure that the transitions and conditions between states were well specified, the FSM was first designed as shown in Fig. 2. CALIBRATING is the state where the robot gathers 500 gyroscope samples and calculates the offset to eliminate drift, WAITING is the state where the robot waits for the start signal, MOVING is the state where the robot moves forward while reading the distance to the wall, TURNING is the state where the robot turns right guided by the gyroscope until the angle reaches the threshold zone, and STOP is the state that stops the robot after completing 8 turns.

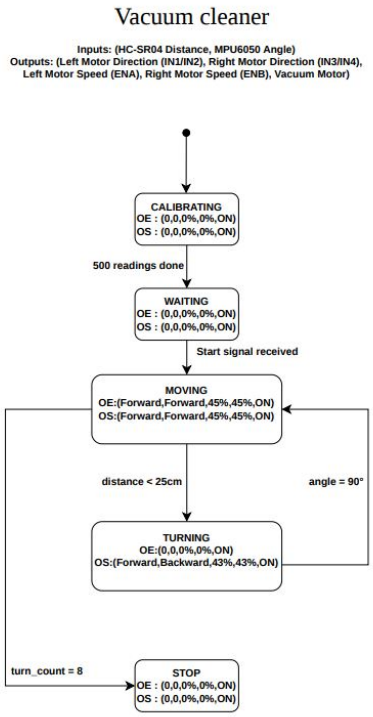


Fig. 2. Finite State Machine designed prior to implementation

C. Web Dashboard and Camera Integration

A web dashboard with multiple pages was designed and implemented using HTML, CSS and JavaScript directly served from the Raspberry Pi [1] via HTTP. There are five pages on this dashboard: Dashboard, Live Camera, Sensor Data, Mission Log, and Settings. An animated step indicator loading screen was added to give a professional appearance. All dashboard features were made mobile responsive. Start and Stop buttons were added so the robot could be controlled from within the browser without access to a terminal. To address button lag due to blocking requests, a threaded HTTP server using ThreadingMixIn was created [7]. The Pi Camera Module 3 with the imx708 sensor was installed and integrated using the Picamera2 library [3], and a live MJPEG stream was added accessible at `/stream`. The dashboard interface is shown in Fig. 3 and Fig. 4.



Fig. 3. Web dashboard main page showing robot status and controls

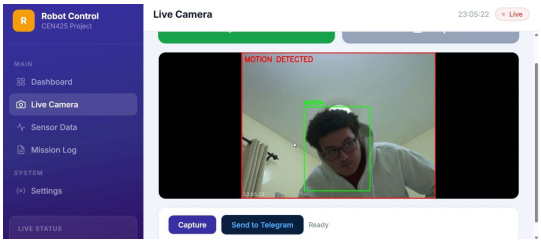


Fig. 4. Web dashboard live camera page with motion detection active

D. Computer Vision and Telegram Bot

The camera stream was integrated with OpenCV [6] to enable real time motion detection using the MOG2 background subtractor algorithm. Morphological operations were applied to filter out noise from the foreground mask, and contour detection was used to draw bounding boxes around detected motion regions. When motion is detected a red border alert and MOTION DETECTED overlay are added to the stream as shown in Fig. 5 and Fig. 6.

The motion detection status is updated in real time on the dashboard. A Telegram bot [4] was included which sends automated notifications including system online alerts, turn completion messages, mission complete alerts, automatic security photos every five seconds, and photos triggered on



Fig. 5. Motion detection active showing green bounding boxes and MOTION DETECTED overlay

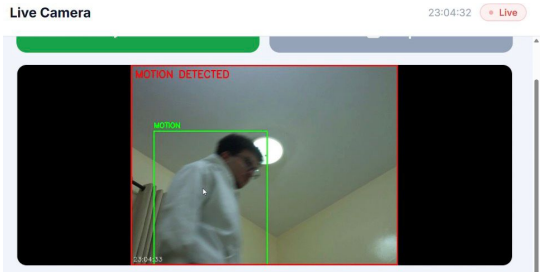


Fig. 6. Motion detection showing red border alert when motion is detected

wall detection as shown in Fig. 7 and Fig. 8. The bot runs on a separate daemon thread [7] so it does not block the robot navigation loop. All Telegram messages use HTML bold formatting for clarity and include a timestamp, robot status, turn count, and distance reading at the time of sending. The security camera feature operates independently from the robot navigation, meaning photos are sent to Telegram even when the robot is not running, making it useful as a standalone security monitoring tool. The interval between automatic security photos was set to five seconds after testing different intervals and finding that five seconds provides a good balance between monitoring frequency and network bandwidth usage. Each photo is saved locally on the Raspberry Pi with a timestamped filename before being sent to Telegram, ensuring that a local backup of all captured images is maintained throughout the session. The cleaning report feature was also implemented as part of this contribution, accessible from the dashboard at the /report endpoint. The report displays the total number of cleaning sessions completed during the day, total laps, total turns, number of photos taken, and number of motion events detected. The user can download the report as a PDF directly from the browser using the built in print function, providing a simple and practical summary of the robot activity for the day. This feature was designed to be lightweight and requires no additional libraries as it generates a plain HTML page served directly from the Python HTTP server. The dashboard itself was designed with a purple and dark color theme and includes a loading screen with five animated steps that simulate the system initialization process, giving the interface a professional appearance when first opened on any device. All dashboard pages update automatically every 600 milliseconds by polling the /status endpoint, which returns a JSON object containing the current robot state, turn count, lap number, distance reading, and mission completion flag.

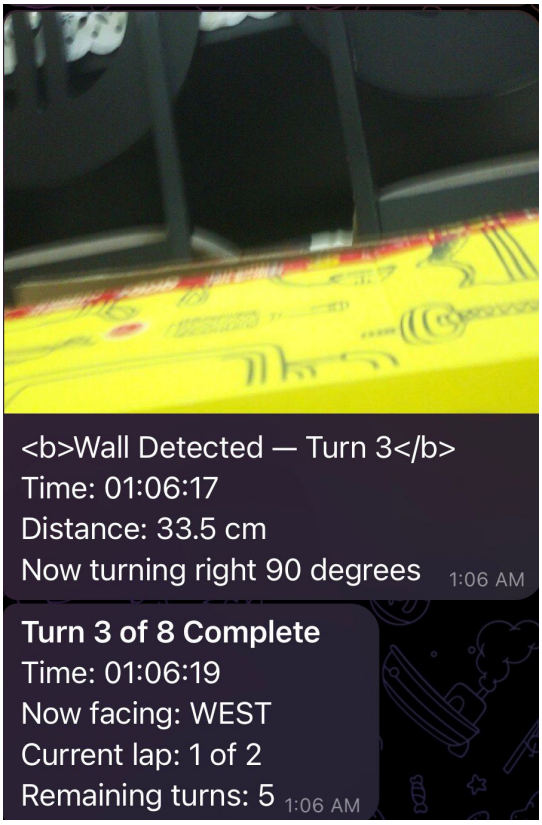


Fig. 7. Telegram bot status messages showing turn completion and mission updates



Fig. 8. Automatic camera photos sent to Telegram every five seconds

E. 3D Design

All the robot chassis parts were designed using Autodesk Inventor Professional as shown in Fig. 9. This comprised the main chassis frame, camera mount holder for the Pi Camera Module 3, and sensor bracket for the HC-SR04 ultrasonic sen-

sor. To achieve the best design, two materials were combined: the chassis was made from PLA filament which allows for easy printing and good rigidity, while parts with higher strength and durability requirements such as the sensor bracket and camera mount were made from carbon fibre reinforced filament. All parts were exported in STL format, 3D printed, and physically assembled onto the robot frame as shown in Fig. 1.

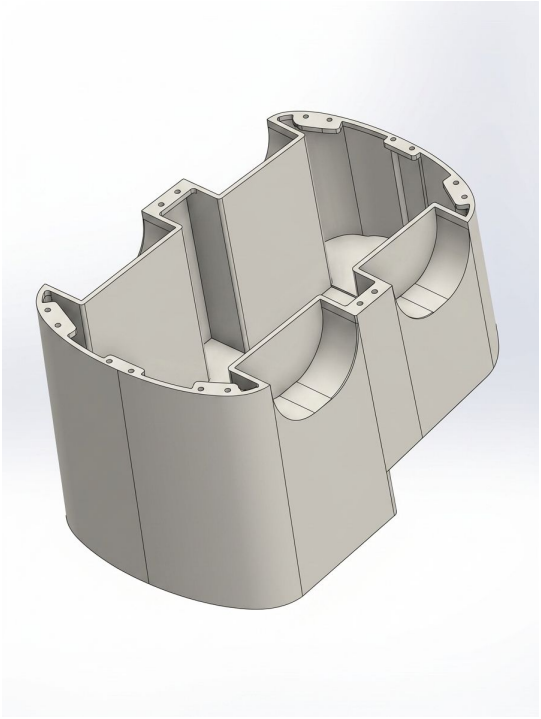


Fig. 9. 3D model of robot chassis designed in Autodesk Inventor Professional

III. Technical Challenges

A number of technical difficulties emerged while implementing the software and system integration contribution, each necessitating a distinct diagnostic method and remedy.

The first major challenge was that the RPi.GPIO library on the Raspberry Pi [1] Bookworm operating system produced completely flawed PWM output. It was not possible to spin the motors properly and the PWM signal was not steady. After research it was found that RPi.GPIO PWM is not fully supported on newer Raspberry Pi models running Bookworm. The solution was to use the native hardware PWM capability with the help of the lgpio library [2], which supports the Raspberry Pi 4.

One other problem was that when the motors were physically wired and tested, the left and right motors were rotating in the wrong direction relative to the chassis orientation. The problem was resolved in software by swapping the IN1 and IN2 signals for the left motor and IN3 and IN4 signals for the right motor. Multiple test runs were necessary to confirm the correct forward direction.

Another difficulty was that the initial turn implementation used a single target angle as the stopping condition. The robot kept overshooting the target because of the time it took to read the gyroscope and stop the motors. The solution was to use a threshold zone between 86.25 and 88 degrees instead of a single target angle. When the measured angle falls into this zone the robot stops immediately, producing a more accurate turn.

Furthermore, the common Python MPU6050 [5] libraries available on GitHub resulted in import errors and incorrect readings on the Raspberry Pi 4 running Bookworm OS. It was decided to bypass any external library and communicate directly with the MPU6050 at address 0x68 using the smbus2 library. This method was more effective and gave full flexibility over sensor configuration.

In addition, when the Start and Stop buttons were introduced in the web dashboard, pressing them caused lag and the dashboard would become unresponsive. The problem was that the standard Python HTTPServer was single threaded and could only handle one request at a time. Button requests were queued and delayed while the server was busy processing the camera stream. The solution was to replace it with a threaded version using ThreadingMixIn [7] from the socketserver module, whereby each request would be processed simultaneously in a separate thread.

The Pi Camera Module 3 did not work on the Raspberry Pi 4 initially. It was determined that the wrong ribbon cable was being used. The Raspberry Pi 5 uses a standard to mini FFC connector while the Raspberry Pi 4 requires a standard to standard FFC connector. After replacing with the correct 15 pin standard both ends cable, both the camera and the stream worked as expected.

Lastly, integrating OpenCV [6] into the camera stream for motion detection led to conflicts between the frame processing thread and the camera capture thread. Both threads were operating on the camera hardware concurrently resulting in corrupted frames and crashes. The problem was overcome by using a threading lock named camera_lock to ensure that only one thread accesses the camera at a time.

IV. Engineering Process

Rather than working on the system as a whole, the software contribution went through a well-defined and iterative engineering process.

The approach started with an on-paper design of the Finite State Machine prior to any coding work being done. All states, transitions, and conditions were clearly defined beforehand, providing a clear roadmap for the entire codebase. This allowed for structured coding and made it easier to locate individual sections of the system for debugging when an error in behavior was discovered.

Each subsystem was then developed and tested separately before being connected to the other subsystems. Initially, the motor control functions were tested with a dedicated test script designed to drive the motors forward for a specified duration. After ensuring the motor direction and speed were correct, the ultrasonic sensor was added and tested independently and distance readings were verified. Gyroscope calibration was then performed and the turn function was tested in isolation

with angle readings printed to the terminal in real time to accurately tune the threshold zone.

The web dashboard was developed separately from the robot navigation code and was tested by running the server independently and accessing it from a laptop and phone browser. Before adding the motion detection layer, the camera stream was integrated and verified. A build and verify cycle was carried out for each addition prior to the introduction of the next one throughout development.

During all development, the SSH terminal was a key debugging tool. Print statements were added at key points in the code to monitor distance readings, gyroscope angle values, FSM state transitions, and motor commands in real time. This enabled quick detection and allowed issues to be identified and isolated without any additional debugging hardware.

Physical testing was performed on the actual robot at each stage. Motor direction was confirmed by visually checking the robot on the floor. The accuracy of the turn function was verified by repeating the turn multiple times and using a physical protractor to measure the actual angle turned. The wall detection threshold was adjusted by placing the robot at various distances from a wall and checking the sensor readings in the terminal.

V. Lessons Learned

The planning and implementation of the CEN425 Smart Vacuum Robot yielded many useful technical and engineering insights that could not be gained from theoretical coursework alone.

The most important lesson learned was that hardware compatibility cannot be assumed without physical testing. The RPi.GPIO library was assumed to work correctly based on online documentation, but it failed completely on the Raspberry Pi 4 [1] running Bookworm. This experience showed the importance of verifying every library and tool on the actual target hardware before implementing any functionality on top of them.

Another important point was that designing the system architecture in advance of coding greatly limits debugging time. Because the Finite State Machine had been drawn on paper before implementation, it was easy to determine which state the robot was in at any given time during testing. Without this structured approach, debugging the robot behavior would have been much harder and more time consuming.

Threading is a fundamental concept in constructing responsive embedded systems [7]. The single threaded HTTP server resulted in lag of the dashboard buttons since it could not handle multiple requests at once. This problem would not have arisen in a purely software environment, and understanding the interaction between threads and shared resources in Python proved to be key in solving this problem.

Working with physical hardware gave insight into never taking software behavior for granted, as it does not necessarily correspond to physical behavior. Motor directions had to be tested, turn angles needed to be iteratively tuned, and sensor thresholds had to be adjusted. Values that appeared correct in theory often required adjustment after observing the actual robot behavior.

Last but not least, integrating computer vision [6] into a resource constrained system like the Raspberry Pi 4 requires careful management of shared resources. The camera lock conflict demonstrated that accessing hardware simultaneously must be explicitly handled through synchronization mechanisms such as threading locks, otherwise data operations with the hardware may lead to unpredictable system behavior.

VI. Possible Improvements

While the system successfully achieved all of the required project objectives, there were some realistic improvements that could be made to enhance the robot's performance and capabilities in future iterations.

The first improvement that could be done is to add a PID controller to the robot's forward motion. Currently the robot moves forward at a constant PWM setting without compensating for drift caused by motor differences or uneven floor surfaces. A PID controller would make continual adjustments to its heading, keeping the robot on a straight path without requiring manual speed tuning.

Another improvement would be to save the gyroscope calibration offset to a file on the Raspberry Pi so that the robot does not have to remain completely still for three seconds on each startup. Calibration could be performed once, saved, and then loaded automatically on subsequent runs, reducing startup time and improving the overall user experience.

Another suggestion for the web dashboard is to include a graph on the sensor data page that plots ultrasonic distance readings over time. This would provide a much clearer visual representation of the robot approaching and detecting walls compared to a single static number displayed on screen.

The motion detection system could also be improved by sending a Telegram photo alert [4] based on motion events rather than only on fixed time intervals. This would make the security monitoring feature more responsive and practical for real world use.

Finally, the chassis could be redesigned so that all cables are routed into the interior of the 3D printed frame. Currently wires are not routed inside the robot, making it look less professional and increasing the risk of wires becoming caught during operation.

VII. Transparent Use of AI

Claude AI by Anthropic was one of various resources utilised during the CEN425 Smart Vacuum Robot project. The AI was applied in a restricted way, such as determining syntax for certain Python functions, giving quick clarification on library usage, and generally offering suggestions for writing this report.

It is important to note that the AI output was not accepted without critical evaluation. In some instances, the suggestions provided by the AI were incorrect for the actual hardware and had to be modified substantially or discarded completely. For example, the initial motor control code suggested using

the RPi.GPIO library which was found to be incompatible with the Raspberry Pi 4 running Bookworm OS after physical testing. The decision to switch to the lgpio library [2] was reached independently through research and physical testing. In the same way, the gyroscope turn threshold values could not be directly taken from any AI suggestion and required several physical test runs to find the correct settings.

The key engineering decisions such as the FSM design, the threading architecture, the motion detection parameters, and the 3D design choices were made independently based on physical experimentation, research, and personal judgment rather than AI generated output.

A comprehensive export of the AI conversation has been submitted alongside this report as required by the course guidelines, as a means of transparency regarding the extent to which AI assistance was used.

REFERENCES

[1] Raspberry Pi Foundation, "Raspberry Pi 4 Model B," Raspberry Pi Ltd., Cambridge, UK, 2019. [Online]. Available: https://www.raspberrypi.com/products/raspberry-pi-4-model-b/

[2] P. Whelan, "lgpio: A library for Linux SBCs," 2021. [Online]. Available: http://abyz.me.uk/lg/lgpio.html

[3] Raspberry Pi Foundation, "Picamera2 Library," 2023. [Online]. Available: https://datasheets.raspberrypi.com/camera/picamera2-manual.pdf

[4] Telegram FZ-LLC, "Telegram Bot API," 2023. [Online]. Available: https://core.telegram.org/bots/api

[5] InvenSense Inc., "MPU-6050 Product Specification Revision 3.4," San Jose, CA, USA, 2013.

[6] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000.

[7] Python Software Foundation, "threading — Thread-based parallelism," 2023. [Online]. Available: https://docs.python.org/3/library/threading.html

APPENDIX

The complete Python source code for the Smart Vacuum Robot navigation system, web dashboard, computer vision motion detection, and Telegram bot integration is provided below.

```python
import io
import lgpio
import time
import smbus2
import threading
import json
import os
import requests
import cv2
import numpy as np
from picamera2 import Picamera2
from http.server import BaseHTTPRequestHandler, HTTPServer
from socketserver import ThreadingMixIn
from datetime import datetime

#        Config

PI_IP             = "192.168.137.100"
PORT              = 8888
PHOTO_DIR         = os.path.expanduser("~/robot/photos")
SECURITY_INTERVAL = 5
os.makedirs(PHOTO_DIR, exist_ok=True)

#        Telegram

TELEGRAM_TOKEN   = "<REDACTED>"
TELEGRAM_CHAT_ID = "<REDACTED>"

def tg_message(text):
    try:
        url = f"https://api.telegram.org/bot{TELEGRAM_TOKEN}/sendMessage"
        requests.post(url, data={
            "chat_id"    : TELEGRAM_CHAT_ID,
            "text"       : text,
            "parse_mode" : "HTML"
        }, timeout=5)
    except Exception as e:
        print(f"Telegram error: {e}")

def tg_photo(path, caption):
    try:
        url = f"https://api.telegram.org/bot{TELEGRAM_TOKEN}/sendPhoto"
        with open(path, 'rb') as f:
            requests.post(url, data={
                "chat_id" : TELEGRAM_CHAT_ID,
                "caption" : caption
            }, files={"photo": f}, timeout=10)
    except Exception as e:
        print(f"Telegram photo error: {e}")

#        Pin Definitions

IN1  = 17
IN2  = 27
IN3  = 22
IN4  = 5
ENA  = 12
ENB  = 13
TRIG = 24
ECHO = 23

#        Settings

LEFT_SPEED  = 45
RIGHT_SPEED = 45
TURN_SPEED  = 43
WALL_CM     = 25
ANGLE_MIN   = 84.8
ANGLE_MAX   = 88
TOTAL_TURNS = 8
PWM_FREQ    = 1000

#        lgpio Setup

h = lgpio.gpiochip_open(0)
for pin in [IN1, IN2, IN3, IN4, ENA, ENB, TRIG]:
    lgpio.gpio_claim_output(h, pin, 0)
lgpio.gpio_claim_input(h, ECHO)

#        MPU6050

MPU_ADDR = 0x68
bus      = smbus2.SMBus(1)

def mpu_write(reg, val):
    bus.write_byte_data(MPU_ADDR, reg, val)

def read_gz():
    high = bus.read_byte_data(MPU_ADDR, 0x47)
    low  = bus.read_byte_data(MPU_ADDR, 0x48)
    val  = (high << 8) | low
    if val > 32767:
        val -= 65536
    return val

mpu_write(0x6B, 0x00); time.sleep(0.1)
mpu_write(0x1B, 0x08); time.sleep(0.1)

#        Calibrate

print("="*40)
print("   SMART VACUUM ROBOT     CEN425")
print("="*40)
print("Calibrating     keep STILL 3 seconds...")
total = 0
for i in range(500):
    total += read_gz()
    time.sleep(0.006)
offset = total / 500.0
print(f"Offset: {offset:.2f}")
print("Calibration done!")
print("="*40)

#        Camera

camera = Picamera2()
camera.configure(camera.create_video_configuration(
    main={"size": (640, 480)}
))
camera.start()
time.sleep(2)
print("Camera ready")

#        Shared State

robot_status = {
    "status"     : "WAITING",
    "turn_count" : 0,
    "lap"        : 1,
    "facing"     : "NORTH",
    "distance"   : 0,
    "mission"    : False
}
status_lock   = threading.Lock()
camera_lock   = threading.Lock()
mission_log   = []
robot_running = False
robot_stop    = False
start_event   = threading.Event()

#        Session Report Data

session_report = {
    "date"         : datetime.now().strftime('%Y-%m-%d'),
    "day"          : datetime.now().strftime('%A'),
    "sessions"     : [],
    "total_laps"   : 0,
    "total_turns"  : 0,
    "photos_taken" : 0,
    "motion_events": 0,
```

```
}
report_lock = threading.Lock()

def add_session(laps, turns, start_time, end_time):
    with report_lock:
        session_report["sessions"].append({
            "start"  : start_time,
            "end"    : end_time,
            "laps"   : laps,
            "turns"  : turns,
        })
        session_report["total_laps"]  += laps
        session_report["total_turns"] += turns

#        OpenCV Motion Detection

bg_subtractor      = cv2.createBackgroundSubtractorMOG2(
    history=500, varThreshold=50, detectShadows=False)
motion_detected_flag = False
motion_lock2         = threading.Lock()

def capture_frame():
    global motion_detected_flag
    with camera_lock:
        frame = camera.capture_array()

    frame_bgr = cv2.cvtColor(frame, cv2.COLOR_RGB2BGR)
    fg_mask   = bg_subtractor.apply(frame_bgr)

    kernel  = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
    fg_mask = cv2.morphologyEx(fg_mask, cv2.MORPH_OPEN, kernel)
    fg_mask = cv2.dilate(fg_mask, kernel, iterations=2)

    contours, _ = cv2.findContours(fg_mask, cv2.RETR_EXTERNAL,
                                   cv2.CHAIN_APPROX_SIMPLE)
    motion = False
    for cnt in contours:
        if cv2.contourArea(cnt) > 3000:
            x, y, w, hh = cv2.boundingRect(cnt)
            cv2.rectangle(frame_bgr, (x, y), (x+w, y+hh), (0, 255, 0), 2)
            cv2.putText(frame_bgr, "MOTION", (x, y-8),
                cv2.FONT_HERSHEY_SIMPLEX, 0.55, (0, 255, 0), 2)
            motion = True

    with motion_lock2:
        old = motion_detected_flag
        motion_detected_flag = motion
        if motion and not old:
            with report_lock:
                session_report["motion_events"] += 1

    if motion:
        cv2.rectangle(frame_bgr, (0, 0), (639, 479), (0, 0, 255), 4)
        cv2.putText(frame_bgr, "MOTION DETECTED", (10, 30),
            cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 255), 2)

    ts = datetime.now().strftime('%H:%M:%S')
    cv2.putText(frame_bgr, ts, (5, 470),
        cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 1)

    _, jpeg = cv2.imencode('.jpg', frame_bgr,
                           [cv2.IMWRITE_JPEG_QUALITY, 80])
    return jpeg.tobytes()

def save_photo(label="photo"):
    with camera_lock:
        filename = datetime.now().strftime('%Y%m%d_%H%M%S') + f'_{label}.jpg'
        filepath = os.path.join(PHOTO_DIR, filename)
        camera.capture_file(filepath)
    with report_lock:
        session_report["photos_taken"] += 1
    return filename, filepath

def take_photo():
    filename, filepath = save_photo("manual")
    mission_log.append({
        "time"  : datetime.now().strftime('%H:%M:%S'),
        "event" : "Photo captured",
        "detail": filename
    })
    caption = (
        f"<b>Photo Captured</b>\n"
        f"Time: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}\n"
        f"File: {filename}\n"
        f"Source: CEN425 Robot Camera"
    )
    threading.Thread(target=tg_photo, args=(filepath, caption),
        daemon=True).start()
    return filename

#        Security Camera Thread

def security_camera_thread():
    print(f"Security camera active     photo every {SECURITY_INTERVAL}s")
    time.sleep(5)
    photo_count = 0
    while True:
        try:
            photo_count += 1
            filename, filepath = save_photo("security")
            with status_lock:
                status = robot_status["status"]
                turns  = robot_status["turn_count"]
                dist   = robot_status["distance"]
            caption = (
                f"<b>Security Camera     Auto Snapshot #{photo_count}</b>\n"
                f"Time: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}\n"
                f"Robot status: {status}\n"
                f"Turns completed: {turns} of 8\n"
                f"Distance to wall: {dist} cm\n"
                f"Location: CEN425 Arena"
            )
            threading.Thread(target=tg_photo, args=(filepath, caption),
                daemon=True).start()
            mission_log.append({
                "time"  : datetime.now().strftime('%H:%M:%S'),
                "event" : "Security photo sent",
                "detail": f"Auto #{photo_count}"
            })
            print(f"Security photo #{photo_count} sent")
        except Exception as e:
            print(f"Security camera error: {e}")
        time.sleep(SECURITY_INTERVAL)

#        PWM

def set_pwm(pin, duty):
    try:
        if duty == 0:
            lgpio.tx_pwm(h, pin, PWM_FREQ, 0)
            lgpio.gpio_write(h, pin, 0)
        else:
            lgpio.tx_pwm(h, pin, PWM_FREQ, duty)
    except:
        pass

#        Motor Functions

def stop_motors():
    try:
        set_pwm(ENA, 0)
        set_pwm(ENB, 0)
        lgpio.gpio_write(h, IN1, 0)
        lgpio.gpio_write(h, IN2, 0)
        lgpio.gpio_write(h, IN3, 0)
        lgpio.gpio_write(h, IN4, 0)
    except:
        pass

def move_forward():
    lgpio.gpio_write(h, IN1, 0)
    lgpio.gpio_write(h, IN2, 1)
    lgpio.gpio_write(h, IN3, 1)
    lgpio.gpio_write(h, IN4, 0)
    set_pwm(ENA, LEFT_SPEED)
    set_pwm(ENB, RIGHT_SPEED)

def soft_start():
    lgpio.gpio_write(h, IN1, 0)
    lgpio.gpio_write(h, IN2, 1)
    lgpio.gpio_write(h, IN3, 1)
    lgpio.gpio_write(h, IN4, 0)
    for i in range(1, 21):
        set_pwm(ENA, (LEFT_SPEED  * i) // 20)
        set_pwm(ENB, (RIGHT_SPEED * i) // 20)
        time.sleep(0.01)

def turn_right_90():
    angle     = 0.0
    last_time = time.time()
    start     = time.time()
    print("Turning RIGHT...")

    with status_lock:
        robot_status["status"] = "TURNING"

    lgpio.gpio_write(h, IN1, 0)
    lgpio.gpio_write(h, IN2, 1)
    lgpio.gpio_write(h, IN3, 0)
    lgpio.gpio_write(h, IN4, 1)
    set_pwm(ENA, TURN_SPEED)
    set_pwm(ENB, TURN_SPEED)

    while True:
        if robot_stop:
            stop_motors()
            break
        gz  = read_gz()
        now = time.time()
        dt  = now - last_time
        last_time = now
        dps    = abs((gz - offset) / 65.5)
        angle += dps * dt
        print(f"  Angle: {angle:.1f}  ", end="\r")
        if ANGLE_MIN <= angle <= ANGLE_MAX:
            stop_motors()
            print(f"\nLOCKED at: {angle:.1f}   PERFECT!")
            break
        if angle > ANGLE_MAX:
            stop_motors()
            print(f"\nOvershot: {angle:.1f}  ")
            break
        if time.time() - start > 6:
            stop_motors()
            print("\nTIMEOUT")
            break
        time.sleep(0.004)

    time.sleep(0.3)

#        Distance

def get_distance():
    try:
        lgpio.gpio_write(h, TRIG, 0)
        time.sleep(0.00004)
        lgpio.gpio_write(h, TRIG, 1)
        time.sleep(0.00001)
        lgpio.gpio_write(h, TRIG, 0)
        timeout = time.time() + 0.03
        while lgpio.gpio_read(h, ECHO) == 0:
            if time.time() > timeout:
```

```
                return 999
        start_time = time.time()
        timeout = time.time() + 0.03
        while lgpio.gpio_read(h, ECHO) == 1:
            if time.time() > timeout:
                return 999
        end_time = time.time()
        return round(((end_time - start_time) * 34300) / 2, 1)
    except:
        return 999

#        Robot Navigation

def run_robot():
    global robot_running, robot_stop
    turn_count = 0
    facing     = 0
    directions = ["NORTH", "EAST", "SOUTH", "WEST"]
    start_time = datetime.now().strftime('%H:%M:%S')

    with status_lock:
        robot_status["status"]     = "MOVING"
        robot_status["turn_count"] = 0
        robot_status["lap"]        = 1
        robot_status["mission"]    = False

    mission_log.append({
        "time"  : datetime.now().strftime('%H:%M:%S'),
        "event" : "Robot started",
        "detail": "Mission begins"
    })

    threading.Thread(target=tg_message, args=(
        f"<b>Robot Started</b>\n"
        f"Time: {datetime.now().strftime('%H:%M:%S')}\n"
        f"Mission: 2 laps, 8 right turns\n"
        f"Wall detection: {WALL_CM} cm\n"
        f"Status: Moving forward",
    ), daemon=True).start()

    soft_start()

    try:
        while turn_count < TOTAL_TURNS:
            if robot_stop:
                break

            move_forward()
            dist = get_distance()

            with status_lock:
                robot_status["distance"]   = dist
                robot_status["facing"]     = directions[facing]
                robot_status["turn_count"] = turn_count
                robot_status["lap"]        = ((turn_count) // 4) + 1
                robot_status["status"]     = "MOVING"

            print(f"Dist:{dist}cm | {directions[facing]} | Turn:{turn_count}/8")

            if 0 < dist < WALL_CM:
                print(f"WALL at {dist}cm     STOPPING")
                stop_motors()

                with status_lock:
                    robot_status["status"] = "STOPPED"

                try:
                    filename, filepath = save_photo(f"wall_turn{turn_count+1}")
                    threading.Thread(target=tg_photo, args=(
                        filepath,
                        f"<b>Wall Detected     Turn {turn_count+1}</b>\n"
                        f"Time: {datetime.now().strftime('%H:%M:%S')}\n"
                        f"Distance: {dist} cm\n"
                        f"Now turning right 90 degrees"
                    ), daemon=True).start()
                except Exception as e:
                    print(f"Wall photo error: {e}")

                time.sleep(0.15)
                turn_right_90()

                turn_count += 1
                facing = (facing + 1) % 4
                lap    = ((turn_count - 1) // 4) + 1

                with status_lock:
                    robot_status["turn_count"] = turn_count
                    robot_status["facing"]     = directions[facing]
                    robot_status["lap"]        = lap
                    robot_status["status"]     = "MOVING"

                mission_log.append({
                    "time"  : datetime.now().strftime('%H:%M:%S'),
                    "event" : f"Turn {turn_count} complete",
                    "detail": f"Now facing {directions[facing]} | Lap {lap}"
                })

                threading.Thread(target=tg_message, args=(
                    f"<b>Turn {turn_count} of 8 Complete</b>\n"
                    f"Time: {datetime.now().strftime('%H:%M:%S')}\n"
                    f"Now facing: {directions[facing]}\n"
                    f"Current lap: {lap} of 2\n"
                    f"Remaining turns: {8 - turn_count}",
                ), daemon=True).start()

                print(f"Turn:{turn_count}/8 | Lap:{lap}/2 | {directions[facing]}")
                print("[" + "#"*turn_count + "."*(8-turn_count) + "]")

                if turn_count < TOTAL_TURNS and not robot_stop:
                    time.sleep(0.2)
                    soft_start()
            else:
                time.sleep(0.03)

        stop_motors()
        end_time = datetime.now().strftime('%H:%M:%S')

        if not robot_stop:
            laps_done = turn_count // 4
            add_session(laps_done, turn_count, start_time, end_time)

            with status_lock:
                robot_status["status"]  = "DONE"
                robot_status["mission"] = True

            mission_log.append({
                "time"  : datetime.now().strftime('%H:%M:%S'),
                "event" : "Mission Complete",
                "detail": "2 laps     8 turns done!"
            })

            threading.Thread(target=tg_message, args=(
                f"<b>Mission Complete</b>\n"
                f"Time: {datetime.now().strftime('%H:%M:%S')}\n"
                f"Result: 2 laps finished successfully\n"
                f"Total turns: 8 of 8 completed\n"
                f"Status: Robot stopped",
            ), daemon=True).start()

            try:
                filename, filepath = save_photo("mission_complete")
                threading.Thread(target=tg_photo, args=(
                    filepath,
                    f"<b>Mission Complete     Final Photo</b>\n"
                    f"Time: {datetime.now().strftime('%H:%M:%S')}\n"
                    f"Result: 2 laps and 8 turns completed\n"
                    f"Location: CEN425 Arena"
                ), daemon=True).start()
            except:
                pass

            print("="*40)
            print("   MISSION COMPLETE!")
            print("="*40)
        else:
            laps_done = turn_count // 4
            add_session(laps_done, turn_count, start_time, end_time)
            with status_lock:
                robot_status["status"] = "STOPPED"
            mission_log.append({
                "time"  : datetime.now().strftime('%H:%M:%S'),
                "event" : "Robot stopped",
                "detail": f"Stopped by user     {turn_count} turns done"
            })
            threading.Thread(target=tg_message, args=(
                f"<b>Robot Stopped by User</b>\n"
                f"Time: {datetime.now().strftime('%H:%M:%S')}\n"
                f"Turns completed: {turn_count} of 8",
            ), daemon=True).start()

    except Exception as e:
        print(f"Robot error: {e}")
    finally:
        stop_motors()
        robot_running = False
        robot_stop    = False

#        Robot thread

def robot_thread():
    global robot_running, robot_stop
    print("")
    print("="*40)
    print(f"  Dashboard: http://{PI_IP}:{PORT}")
    print("  Press Start from dashboard or phone")
    print("="*40)
    while True:
        start_event.wait()
        start_event.clear()
        if robot_running:
            run_robot()

#        Generate HTML Report

def generate_report_html():
    with report_lock:
        r = dict(session_report)

    sessions_html = ""
    for i, s in enumerate(r["sessions"], 1):
        sessions_html += f"""
        <tr>
            <td>{i}</td>
            <td>{s['start']}</td>
            <td>{s['end']}</td>
            <td>{s['laps']}</td>
            <td>{s['turns']}</td>
        </tr>"""

    if not sessions_html:
        sessions_html = "<tr><td colspan='5'>No sessions recorded yet</td></tr>"

    return f"""<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title>Cleaning Report     {r['date']}</title>
<style>
body{{font-family:Arial,sans-serif;padding:30px;max-width:700px;margin:auto;color:#1e293b;}}
h1{{color:#3730a3;margin-bottom:4px;}}
.subtitle{{color:#64748b;font-size:14px;margin-bottom:24px;}}
.stats{{display:grid;grid-template-columns:repeat(2,1fr);gap:16px;margin-bottom:28px;}}
```

```
.stat{{background:#f1f5f9;border-radius:10px;padding:16px;text-align:center;}}
.stat-val{{font-size:32px;font-weight:800;color:#3730a3;}}
.stat-label{{font-size:12px;color:#64748b;margin-top:4px;}}
table{{width:100%;border-collapse:collapse;margin-bottom:24px;}}
th{{background:#3730a3;color:white;padding:10px;font-size:13px;text-align:left;}}
td{{padding:9px 10px;border-bottom:1px solid #e2e8f0;font-size:13px;}}
tr:last-child td{{border-bottom:none;}}
.footer{{font-size:11px;color:#94a3b8;text-align:center;margin-top:20px;}}
.btn{{display:block;margin:20px auto;padding:12px
      28px;background:#3730a3;color:white;
     border:none;border-radius:8px;font-size:14px;font-weight:600;cursor:pointer;
     text-align:center;width:fit-content;}}
@media print{{.btn{{display:none;}}}}
</style>
</head>
<body>
<h1>Smart Vacuum Robot</h1>
<div class="subtitle">Cleaning Report     {r['day']}, {r['date']}</div>
<div class="stats">
  <div class="stat"><div class="stat-val">{len(r['sessions'])}</div><div
       class="stat-label">Sessions Today</div></div>
  <div class="stat"><div class="stat-val">{r['total_laps']}</div><div
       class="stat-label">Total Laps</div></div>
  <div class="stat"><div class="stat-val">{r['total_turns']}</div><div
       class="stat-label">Total Turns</div></div>
  <div class="stat"><div class="stat-val">{r['photos_taken']}</div><div
       class="stat-label">Photos Taken</div></div>
  <div class="stat"><div class="stat-val">{r['motion_events']}</div><div
       class="stat-label">Motion Events</div></div>
  <div class="stat"><div class="stat-val">{r['date']}</div><div
       class="stat-label">Report Date</div></div>
</div>
<h3>Session Log</h3>
<table>
  <tr><th>#</th><th>Start</th><th>End</th><th>Laps</th><th>Turns</th></tr>
  {sessions_html}
</table>
<button class="btn" onclick="window.print()">Download as PDF</button>
<div class="footer">Generated by CEN425 Smart Vacuum Robot     Abu Dhabi
      University<br>
{datetime.now().strftime('%Y-%m-%d %H:%M:%S')}</div>
</body>
</html>"""

#        HTML Dashboard

HTML = f'''<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1,maximum-scale=1">
<title>Robot Control     CEN425</title>
<link
      href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&disp
      rel="stylesheet">
<style>
*,*::before,*::after{{box-sizing:border-box;margin:0;padding:0;}}
body{{font-family:'Inter',-apple-system,sans-serif;background:#f1f5f9;color:#1e293b;
#loader{{position:fixed;inset:0;background:#312e81;z-index:999;display:flex;flex-direc
.ld-title{{font-size:22px;font-weight:700;color:white;margin-bottom:4px;text-align:cent
.ld-sub{{font-size:12px;color:rgba(255,255,255,0.55);margin-bottom:36px;text-align:c
.ld-steps{{width:100%;max-width:300px;margin-bottom:28px;display:flex;flex-direction:
.ld-step{{display:flex;align-items:center;gap:10px;opacity:0;transition:opacity
      0.3s;}}
.ld-step.visible{{opacity:1;}}
.ld-check{{width:20px;height:20px;border-radius:50%;border:2px solid
      rgba(255,255,255,0.3);display:flex;align-items:center;justify-content:center;
      0.3s;}}
.ld-check.done{{border-color:#4ade80;background:rgba(74,222,128,0.2);}}
.ld-check-dot{{width:6px;height:6px;border-radius:50%;background:rgba(255,255,255,0.
.ld-check.done .ld-check-dot{{background:#4ade80;}}
.ld-step-text{{font-size:13px;color:rgba(255,255,255,0.9);}}
.ld-bar-track{{height:4px;background:rgba(255,255,255,0.12);border-radius:2px;overflo
.ld-bar-fill{{height:100%;background:#818cf8;border-radius:2px;width:0;transition:wi
      0.1s;}}
#loader.hiding{{opacity:0;transition:opacity 0.6s;pointer-events:none;}}
@keyframes pulse{{0%,100%{{opacity:1;}}50%{{opacity:0.4;}}}}
#app{{display:none;min-height:100vh;}}
#app.visible{{display:flex;}}
.sidebar{{width:230px;background:linear-gradient(170deg,#3730a3,#312e81
      60%,#2e1065);min-height:100vh;display:flex;flex-direction:column;flex-shrink:0
      0.3s;}}
.sidebar.open{{transform:translateX(0);}}
@media(min-width:768px){{.sidebar{{position:sticky;transform:none;}}.main{{height:100vh
.sidebar-brand{{padding:20px 16px;border-bottom:1px solid
      rgba(255,255,255,0.07);display:flex;align-items:center;gap:11px;}}
.brand-icon{{width:36px;height:36px;background:#f59e0b;border-radius:9px;display:flex;a
.brand-name{{font-size:14px;font-weight:700;color:white;}}
.brand-sub{{font-size:11px;color:rgba(255,255,255,0.45);}}
.sidebar-nav{{padding:12px 10px;flex:1;}}
.nav-item{{display:flex;align-items:center;gap:9px;padding:10px
      12px;border-radius:8px;font-size:13px;font-weight:500;color:rgba(255,255,255,0.5
      0.15s,color 0.15s;margin-bottom:1px;}}
.nav-item:hover{{background:rgba(255,255,255,0.07);color:rgba(255,255,255,0.85);}}
.nav-item.active{{background:rgba(255,255,255,0.13);color:white;}}
.nav-item
      svg{{width:15px;height:15px;stroke:currentColor;fill:none;stroke-width:2;stroke
.sidebar-widget{{margin:10px;padding:14px;background:rgba(255,255,255,0.07);border-r
      solid rgba(255,255,255,0.08);}}
.widget-title{{font-size:10px;font-weight:600;color:rgba(255,255,255,0.35);text-tran
.widget-row{{display:flex;justify-content:space-between;align-items:center;padding:3
      0;}}
.widget-key{{font-size:12px;color:rgba(255,255,255,0.45);}}
.widget-val{{font-size:12px;font-weight:600;color:white;}}
.live-dot{{width:6px;height:6px;border-radius:50%;background:#4ade80;animation:pulse
      1.8s ease-in-out infinite;display:inline-block;margin-right:4px;}}
.main{{flex:1;display:flex;flex-direction:column;min-width:0;min-height:100vh;}}
.topbar{{background:white;border-bottom:1px solid #e2e8f0;padding:0
      16px;height:56px;display:flex;align-items:center;justify-content:space-between;f
.topbar-left{{display:flex;align-items:center;gap:12px;}}
.menu-btn{{width:36px;height:36px;border:none;background:#f1f5f9;border-radius:8px;c
.menu-btn svg{{width:18px;height:18px;stroke:#475569;fill:none;stroke-width:2;}}
@media(min-width:768px){{.menu-btn{{display:none;}}}}
.page-title{{font-size:16px;font-weight:700;color:#0f172a;}}
.topbar-right{{display:flex;align-items:center;gap:10px;}}
.topbar-time{{font-size:12px;color:#94a3b8;}}
.live-badge{{display:flex;align-items:center;gap:5px;background:#fef2f2;border:1px
      solid #fecaca;border-radius:20px;padding:3px
      10px;font-size:11px;font-weight:600;color:#ef4444;}}
.page{{display:none;flex:1;overflow-y:auto;padding:16px;}}
.page.active{{display:block;}}
@media(min-width:768px){{.page{{padding:22px 24px;}}}}
.control-row{{display:grid;grid-template-columns:1fr
      1fr;gap:12px;margin-bottom:14px;}}
.btn-start{{padding:16px;background:#16a34a;color:white;border:none;border-radius:12px;font-size:15p
.btn-start:disabled{{background:#94a3b8;cursor:not-allowed;}}
.btn-stop{{padding:16px;background:#dc2626;color:white;border:none;border-radius:12px;font-size:15px;
.btn-stop:disabled{{background:#94a3b8;cursor:not-allowed;}}
.btn-report{{width:100%;padding:14px;background:#0f172a;color:#f59e0b;border:2px
      solid
      #f59e0b;border-radius:12px;font-size:14px;font-weight:700;font-family:inherit;cursor:pointer;m
.cards-row{{display:grid;grid-template-columns:1fr
      1fr;gap:10px;margin-bottom:14px;}}
@media(min-width:768px){{.cards-row{{grid-template-columns:repeat(4,1fr);gap:14px;}}}}
.card{{background:white;border-radius:12px;padding:14px;border:1px solid #e2e8f0;}}
.card-val{{font-size:24px;font-weight:800;color:#0f172a;line-height:1;margin-bottom:2px;}}
.card-label{{font-size:11px;color:#64748b;}}
.card-hint{{font-size:10px;color:#94a3b8;margin-top:4px;}}
.cam-card{{background:white;border-radius:12px;border:1px solid
      #e2e8f0;overflow:hidden;margin-bottom:14px;}}
.cam-card-top{{padding:12px 16px;border-bottom:1px solid
      #f1f5f9;display:flex;align-items:center;justify-content:space-between;}}
.cam-card-title{{font-size:13px;font-weight:600;color:#0f172a;}}
.cam-card-meta{{font-size:11px;color:#94a3b8;}}
.cam-view{{position:relative;background:#0f172a;}}
.cam-view img{{width:100%;display:block;}}
.cam-rec-badge{{position:absolute;top:8px;left:8px;background:rgba(0,0,0,0.65);padding:3px
      7px;border-radius:4px;font-size:10px;font-weight:700;color:#ef4444;display:flex;align-items:ce
.rec-pulse{{width:5px;height:5px;border-radius:50%;background:#ef4444;animation:pulse
      1s infinite;}}
.cam-bottom{{padding:10px 14px;border-top:1px solid
      #f1f5f9;display:flex;align-items:center;gap:8px;flex-wrap:wrap;}}
.btn-primary{{padding:8px
      16px;background:#3730a3;color:white;border:none;border-radius:8px;font-size:12px;font-weight:6
.btn-tg{{padding:8px 16px;background:#0a1f3c;color:#4db8ff;border:1px solid
      #1a3a5a;border-radius:8px;font-size:12px;font-weight:600;font-family:inherit;cursor:pointer;}}
.cap-status{{font-size:11px;color:#64748b;flex:1;}}
.log-table{{background:white;border-radius:12px;border:1px solid
      #e2e8f0;overflow:hidden;}}
.log-header{{display:grid;grid-template-columns:70px 1fr 1fr;padding:8px
      14px;background:#f8fafc;border-bottom:1px solid
      #e2e8f0;font-size:10px;font-weight:600;color:#64748b;text-transform:uppercase;}}
.log-row{{display:grid;grid-template-columns:70px 1fr 1fr;padding:9px
      14px;border-bottom:1px solid #f1f5f9;font-size:12px;}}
.log-row:last-child{{border-bottom:none;}}
.log-time{{color:#94a3b8;}}
.log-event{{color:#0f172a;font-weight:500;}}
.log-detail{{color:#64748b;font-size:11px;}}
.log-empty{{padding:32px;text-align:center;color:#94a3b8;font-size:13px;}}
.info-card{{background:white;border-radius:12px;border:1px solid
      #e2e8f0;overflow:hidden;margin-bottom:14px;}}
.info-card-top{{padding:12px 16px;border-bottom:1px solid #f1f5f9;}}
.info-card-title{{font-size:13px;font-weight:600;color:#0f172a;}}
.info-row{{display:flex;justify-content:space-between;align-items:center;padding:8px
      16px;border-bottom:1px solid #f8fafc;font-size:12px;}}
.info-row:last-child{{border-bottom:none;}}
.info-key{{color:#64748b;}}
.info-val{{font-weight:500;color:#0f172a;}}
.overlay{{display:none;position:fixed;inset:0;background:rgba(0,0,0,0.5);z-index:40;}}
.overlay.show{{display:block;}}
</style>
</head>
<body>

<div id="loader">
  <div class="ld-title">Smart Vacuum Robot</div>
  <div class="ld-sub">CEN425     Abu Dhabi University</div>
  <div class="ld-steps">
    <div class="ld-step" id="ls0"><div class="ld-check" id="lc0"><div
         class="ld-check-dot"></div></div><span class="ld-step-text">Connecting
         to Raspberry Pi 4</span></div>
    <div class="ld-step" id="ls1"><div class="ld-check" id="lc1"><div                transform
         class="ld-check-dot"></div></div><span class="ld-step-text">Initializing
         imx708 camera</span></div>
    <div class="ld-step" id="ls2"><div class="ld-check" id="lc2"><div
         class="ld-check-dot"></div></div><span class="ld-step-text">Calibrating
         MPU6050 gyroscope</span></div>
    <div class="ld-step" id="ls3"><div class="ld-check" id="lc3"><div                ex-shrink:0;}}
         class="ld-check-dot"></div></div><span class="ld-step-text">Testing
         HC-SR04 ultrasonic</span></div>
    <div class="ld-step" id="ls4"><div class="ld-check" id="lc4"><div
         class="ld-check-dot"></div></div><span class="ld-step-text">Starting
         motion detection</span></div>
  </div>
  <div class="ld-bar-track"><div class="ld-bar-fill" id="ld-fill"></div></div>
</div>

<div id="app">
  <div class="overlay" id="overlay" onclick="closeSidebar()"></div>
  <aside class="sidebar" id="sidebar">
    <div class="sidebar-brand">
      <div class="brand-icon">R</div>
      <div><div class="brand-name">Robot Control</div><div
           class="brand-sub">CEN425 Project</div></div>
    </div>
    <nav class="sidebar-nav">
      <div class="nav-item active" onclick="goto('dashboard',this)">
        <svg viewBox="0 0 24 24"><rect x="3" y="3" width="7" height="7"
             rx="1"/><rect x="14" y="3" width="7" height="7" rx="1"/><rect x="3"
             y="14" width="7" height="7" rx="1"/><rect x="14" y="14" width="7"
             height="7" rx="1"/></svg>Dashboard
      </div>
```

```
      <div class="nav-item" onclick="goto('camera',this)">
        <svg viewBox="0 0 24 24"><path d="M23 19a2 2 0 0 1-2 2H3a2 2 0 0 1-2-2V8a2
             2 0 0 1 2-2h4l2-3h6l2 3h4a2 2 0 0 1 2 2z"/><circle cx="12" cy="13"
             r="4"/></svg>Live Camera
      </div>
      <div class="nav-item" onclick="goto('sensors',this)">
        <svg viewBox="0 0 24 24"><polyline points="22 12 18 12 15 21 9 3 6 12 2
             12"/></svg>Sensor Data
      </div>
      <div class="nav-item" onclick="goto('log',this)">
        <svg viewBox="0 0 24 24"><path d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2
             2h12a2 2 0 0 0 2-2V8z"/><polyline points="14 2 14 8 20
             8"/></svg>Mission Log
      </div>
      <div class="nav-item" onclick="goto('settings',this)">
        <svg viewBox="0 0 24 24"><circle cx="12" cy="12" r="3"/><path d="M19.07
             4.93a10 10 0 0 1 0 14.14M4.93 4.93a10 10 0 0 0 0 0
             14.14"/></svg>Settings
      </div>
    </nav>
    <div class="sidebar-widget">
      <div class="widget-title">Live Status</div>
      <div class="widget-row"><span class="widget-key">Connection</span><span
           class="widget-val"><span class="live-dot"></span>Online</span></div>
      <div class="widget-row"><span class="widget-key">Turn</span><span
           class="widget-val" id="wb-turn">0 / 8</span></div>
      <div class="widget-row"><span class="widget-key">Lap</span><span
           class="widget-val" id="wb-lap">1 / 2</span></div>
      <div class="widget-row"><span class="widget-key">Heading</span><span
           class="widget-val" id="wb-head">NORTH</span></div>
    </div>
  </aside>

  <div class="main">
    <div class="topbar">
      <div class="topbar-left">
        <button class="menu-btn" onclick="openSidebar()">
          <svg viewBox="0 0 24 24"><line x1="3" y1="6" x2="21" y2="6"/><line
               x1="3" y1="12" x2="21" y2="12"/><line x1="3" y1="18" x2="21"
               y2="18"/></svg>
        </button>
        <div class="page-title" id="page-title">Dashboard</div>
      </div>
      <div class="topbar-right">
        <span class="topbar-time" id="clock">--:--:--</span>
        <div class="live-badge"><div class="rec-pulse"></div>Live</div>
      </div>
    </div>

    <div class="page active" id="page-dashboard">
      <div class="control-row">
        <button class="btn-start" id="btn-start" onclick="startRobot()">
          <svg viewBox="0 0 24 24"
               style="width:18px;height:18px;stroke:white;fill:none;stroke-width:2.5"
               points="5 3 19 12 5 21 5 3"/></svg>Start Robot
        </button>
        <button class="btn-stop" id="btn-stop" onclick="stopRobot()" disabled>
          <svg viewBox="0 0 24 24"
               style="width:18px;height:18px;stroke:white;fill:none;stroke-width:2.5"
               x="3" y="3" width="18" height="18" rx="2"/></svg>Stop Robot
        </button>
      </div>
      <button class="btn-report" onclick="openReport()">View Cleaning Report
            Download as PDF</button>
      <div class="cards-row">
        <div class="card"><div class="card-val" id="c-turns">0</div><div
             class="card-label">Turns Done</div><div class="card-hint">of 8
             total</div></div>
        <div class="card"><div class="card-val" id="c-lap">1</div><div
             class="card-label">Current Lap</div><div class="card-hint">of 2
             laps</div></div>
        <div class="card"><div class="card-val" id="c-dist"
             style="font-size:20px;">---</div><div class="card-label">Distance
             (cm)</div><div class="card-hint">to nearest wall</div></div>
        <div class="card"><div class="card-val" id="c-status"
             style="font-size:13px;">WAITING</div><div class="card-label">Robot
             Status</div><div class="card-hint" id="c-facing">Heading:
             NORTH</div></div>
      </div>
      <div class="cam-card">
        <div class="cam-card-top"><div class="cam-card-title">Live Feed</div><span
             class="cam-card-meta">imx708    640x480</span></div>
        <div class="cam-view"><img src="/stream" alt="camera"/><div
             class="cam-rec-badge"><div class="rec-pulse"></div>REC</div></div>
        <div class="cam-bottom">
          <button class="btn-primary" onclick="capturePhoto()">Capture</button>
          <button class="btn-tg" onclick="sendToTelegram()">Send to
               Telegram</button>
          <span class="cap-status" id="cap-msg">Ready</span>
        </div>
      </div>
    </div>

    <div class="page" id="page-camera">
      <div class="control-row">
        <button class="btn-start" id="btn-start2" onclick="startRobot()">
          <svg viewBox="0 0 24 24"
               style="width:18px;height:18px;stroke:white;fill:none;stroke-width:2.5"
               points="5 3 19 12 5 21 5 3"/></svg>Start
        </button>
        <button class="btn-stop" id="btn-stop2" onclick="stopRobot()" disabled>
          <svg viewBox="0 0 24 24"
               style="width:18px;height:18px;stroke:white;fill:none;stroke-width:2.5"
               x="3" y="3" width="18" height="18" rx="2"/></svg>Stop
        </button>
      </div>
      <div class="cam-card">
        <div class="cam-view"><img src="/stream" alt="Full camera feed"/></div>
        <div class="cam-bottom">
          <button class="btn-primary" onclick="capturePhoto()">Capture</button>
          <button class="btn-tg" onclick="sendToTelegram()">Send to
               Telegram</button>
          <span class="cap-status" id="cap-msg2">Ready</span>
        </div>
      </div>
    </div>

    <div class="page" id="page-sensors">
      <div class="info-card">
        <div class="info-card-top"><div class="info-card-title">Sensor
             Readings</div></div>
        <div class="info-row"><span class="info-key">Ultrasonic
             (HC-SR04)</span><span class="info-val" id="sr-dist">---
             cm</span></div>
        <div class="info-row"><span class="info-key">Gyroscope
             (MPU6050)</span><span class="info-val"
             id="sr-gyro">Active</span></div>
        <div class="info-row"><span class="info-key">Motion Detection</span><span
             class="info-val" id="sr-motion">Monitoring</span></div>
        <div class="info-row"><span class="info-key">Motor Status</span><span
             class="info-val" id="sr-motor">Waiting</span></div>
        <div class="info-row"><span class="info-key">Current Lap</span><span
             class="info-val" id="sr-lap">1 / 2</span></div>
        <div class="info-row"><span class="info-key">Mission Status</span><span
             class="info-val" id="sr-mission">Waiting to Start</span></div>
      </div>
    </div>

    <div class="page" id="page-log">
      <div class="log-table">
        <div
             class="log-header"><span>Time</span><span>Event</span><span>Detail</span></div>
        <div id="log-body"><div class="log-empty">No events yet     press Start
             Robot to begin</div></div>
      </div>
    </div>

    <div class="page" id="page-settings">
      <div class="info-card">
        <div class="info-card-top"><div class="info-card-title">Robot
             Configuration</div></div>
        <div class="info-row"><span class="info-key">Wall Detection
             Distance</span><span class="info-val">{WALL_CM} cm</span></div>
        <div class="info-row"><span class="info-key">Turn Zone</span><span
             class="info-val">{ANGLE_MIN} to {ANGLE_MAX} degrees</span></div>
        <div class="info-row"><span class="info-key">Left Motor Speed</span><span
             class="info-val">{LEFT_SPEED} / 100</span></div>
        <div class="info-row"><span class="info-key">Right Motor Speed</span><span
             class="info-val">{RIGHT_SPEED} / 100</span></div>
        <div class="info-row"><span class="info-key">Turn Speed</span><span
             class="info-val">{TURN_SPEED} / 100</span></div>
        <div class="info-row"><span class="info-key">Total Turns</span><span
             class="info-val">8</span></div>
        <div class="info-row"><span class="info-key">Security Interval</span><span
             class="info-val">{SECURITY_INTERVAL} seconds</span></div>
        <div class="info-row"><span class="info-key">Server</span><span
             class="info-val">{PI_IP}:{PORT}</span></div>
      </div>
    </div>

  </div>
</div>

<script>
const TOTAL=5000,STEP=TOTAL/5;
let pct=0;
const fill=document.getElementById("ld-fill");
const
      pBar=setInterval(()=>{pct=Math.min(pct+1,100);fill.style.width=pct+"%";if(pct>=100)clearInter
function ldShow(i){document.getElementById("ls"+i).classList.add("visible");}}
function ldDone(i){document.getElementById("lc"+i).classList.add("done");}}
ldShow(0);
for(let
      i=0;i<5;i++){setTimeout(()=>{if(i>0)ldDone(i-1);ldShow(i);if(i===4){setTimeout(()=>{ldDone
function
      openSidebar(){document.getElementById("sidebar").classList.add("open");document.getElementByI
function
      closeSidebar(){document.getElementById("sidebar").classList.remove("open");document.getElemen
const titles={dashboard:"Dashboard",camera:"Live Camera",sensors:"Sensor
      Data",log:"Mission Log",settings:"Settings"};
function
      goto(page,el){document.querySelectorAll(".page").forEach(p=>p.classList.remove("active"));doc
setInterval(()=>{document.getElementById("clock").textContent=new
      Date().toLocaleTimeString("en-GB");},1000);
const logs=[];
function addLog(event,detail){const t=new
      Date().toLocaleTimeString("en-GB");logs.unshift({t,event,detail});document.getElementById("l
      class="log-row"><span class="log-time">${t}</span><span
      class="log-event">${event}</span><span
      class="log-detail">${detail}</span></div>`).join("");}
function setButtons(running){["btn-start","btn-start2"].forEach(id=>{const
      b=document.getElementById(id);if(b)b.disabled=running;});["btn-stop","btn-stop2"].forEach(id=
      b=document.getElementById(id);if(b)b.disabled=!running;});}
function
      startRobot(){fetch("/start").then(r=>r.json()).then(d=>{if(d.status==="started"){setButtons
      started","Mission initiated from
      dashboard");}}else{addLog("Info",d.message||"Already
      running");}}}).catch(()=>addLog("Error","Could not connect"));}}
function
      stopRobot(){fetch("/stop").then(r=>r.json()).then(()=>{setButtons(false);addLog("Robot
      stopped","Stopped from dashboard");}}).catch(()=>addLog("Error","Could not
      stop"));}}
let lastTurn=0,lastStatus="";
function update(){fetch("/status").then(r=>r.json()).then(d=>{const
      tc=d.turn_count||0,lp=d.lap||1,dist=d.distance||0,facing=d.facing||"NORTH",status=d.status||"W
      completed","Turn "+tc+" of 8     facing
      "+facing);if(d.mission&&lastStatus!=="DONE"){addLog("Mission complete","2
      laps
      finished");setButtons(false);}}lastTurn=tc;lastStatus=d.mission?"DONE":status;document.getElem
      / 8";document.getElementById("wb-lap").textContent=lp+" /
      2";document.getElementById("wb-head").textContent=facing;document.getElementById("c-turns").te
      "+facing;document.getElementById("sr-dist").textContent=(dist||"---")+"
```

```
        cm";document.getElementById("sr-gyro").textContent=status==="TURNING"?"Turning
        DETECTED":"Clear";document.getElementById("sr-motor").textContent=d.mission?"
        /
        2";document.getElementById("sr-mission").textContent=d.mission?"Complete":sta
        Progress";})).catch(()=>{{});}}
setInterval(update,600);update();
function
        capturePhoto(){{fetch("/photo").then(r=>r.json()).then(d=>{{document.getEleme
        "+d.filename;addLog("Photo
        captured",d.filename);})).catch(()=>document.getElementById("cap-msg").textCo
function
        sendToTelegram(){{fetch("/telegram_photo").then(r=>r.json()).then(()=>{{docum
        photo sent","Manual
        send");})).catch(()=>document.getElementById("cap-msg").textContent="Failed")
function openReport(){{window.open("/report","_blank");}}
</script>
</body>
</html>'''

#        Threaded Web Server

class ThreadedServer(ThreadingMixIn, HTTPServer):
    pass

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        global robot_running, robot_stop

        if self.path == '/':
            self.send_response(200)
            self.send_header('Content-Type', 'text/html')
            self.end_headers()
            self.wfile.write(HTML.encode())

        elif self.path == '/start':
            self.send_response(200)
            self.send_header('Content-Type', 'application/json')
            self.send_header('Access-Control-Allow-Origin', '*')
            self.end_headers()
            if not robot_running:
                robot_running = True
                robot_stop    = False
                start_event.set()
                print("Robot starting from dashboard!")
                self.wfile.write(json.dumps({"status": "started"}).encode())
            else:
                self.wfile.write(json.dumps({
                    "status": "already_running",
                    "message": "Robot is already running"
                }).encode())

        elif self.path == '/stop':
            self.send_response(200)
            self.send_header('Content-Type', 'application/json')
            self.send_header('Access-Control-Allow-Origin', '*')
            self.end_headers()
            robot_stop = True
            stop_motors()
            with status_lock:
                robot_status["status"] = "STOPPED"
            print("Robot stopped from dashboard!")
            self.wfile.write(json.dumps({"status": "stopped"}).encode())

        elif self.path == '/stream':
            self.send_response(200)
            self.send_header('Content-Type',
                'multipart/x-mixed-replace; boundary=frame')
            self.end_headers()
            try:
                while True:
                    frame = capture_frame()
                    self.wfile.write(b'--frame\r\n')
                    self.wfile.write(b'Content-Type: image/jpeg\r\n\r\n')
                    self.wfile.write(frame)
                    self.wfile.write(b'\r\n')
                    time.sleep(0.05)
            except (BrokenPipeError, ConnectionResetError):
                pass

        elif self.path == '/photo':
            try:
                filename = take_photo()
                self.send_response(200)
                self.send_header('Content-Type', 'application/json')
                self.end_headers()
                self.wfile.write(json.dumps(
                    {"filename": filename, "status": "ok"}
                ).encode())
            except Exception as e:
                self.send_response(500)
                self.end_headers()

        elif self.path == '/telegram_photo':
            try:
                filename, filepath = save_photo("manual_tg")
                caption = (
                    f"<b>Manual Photo     Sent from Dashboard</b>\n"
                    f"Time: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}\n"
                    f"Triggered by: user via web dashboard\n"
                    f"Source: CEN425 Robot Camera"
                )
                threading.Thread(target=tg_photo, args=(filepath, caption),
                      daemon=True).start()
                self.send_response(200)
                self.send_header('Content-Type', 'application/json')
                self.end_headers()
                self.wfile.write(json.dumps({"status": "sent"}).encode())
            except Exception as e:
                self.send_response(500)
                self.end_headers()

        elif self.path == '/report':
            self.send_response(200)
            self.send_header('Content-Type', 'text/html')
            self.end_headers()
            self.wfile.write(generate_report_html().encode())

        elif self.path == '/status':
            self.send_response(200)
            self.send_header('Content-Type', 'application/json')
            self.send_header('Access-Control-Allow-Origin', '*')
            self.end_headers()
            with status_lock:
                s = dict(robot_status)
            with motion_lock2:
                s["motion"] = motion_detected_flag
            self.wfile.write(json.dumps(s).encode())

        else:
            self.send_response(404)
            self.end_headers()

    def log_message(self, format, *args):
        pass

#        Main

if __name__ == '__main__':
    print('='*40)
    print('  Smart Vacuum Robot     CEN425')
    print('='*40)
    print(f'Dashboard : http://{PI_IP}:{PORT}')
    print(f'Report    : http://{PI_IP}:{PORT}/report')
    print(f'Security  : photo every {SECURITY_INTERVAL}s')
    print('Press Start from dashboard or phone')
    print('='*40)

    threading.Thread(target=tg_message, args=(
        f"<b>CEN425 Robot System Online</b>\n"
        f"Dashboard: http://{PI_IP}:{PORT}\n"
        f"Camera: imx708 12MP     Motion detection active\n"
        f"Security: photo every {SECURITY_INTERVAL}s\n"
        f"Status: Ready     press Start from dashboard",
    ), daemon=True).start()

    threading.Thread(target=security_camera_thread, daemon=True).start()
    threading.Thread(target=robot_thread, daemon=True).start()

    server = ThreadedServer(('0.0.0.0', PORT), Handler)
    try:
        server.serve_forever()
    except KeyboardInterrupt:
        camera.stop()
        stop_motors()
        lgpio.gpiochip_close(h)
        print('Stopped.')
```